

This lecture explains the concept of abstraction in object-oriented programming (OOP) using Python. Abstraction involves creating a skeleton or blueprint of a class with methods that are declared but not implemented. These methods are meant to be implemented by other classes that inherit from the abstract class.

Key concepts covered:

- **Abstraction Definition:** Abstraction is creating a skeleton that will be implemented later.
- **Abstract Base Class (ABC):** To achieve abstraction in Python, you need to use the `ABC` class and `abstractmethod` decorator from the `abc` module.
- **Creating an Abstract Class:** An abstract class is created by inheriting from `ABC`. Methods within the class can be declared as abstract using the `@abstractmethod` decorator. These methods do not have an implementation in the abstract class.
- **Implementation in Subclasses:** Subclasses that inherit from the abstract class are responsible for providing concrete implementations for all abstract methods.
- **Enforcing Implementation:** If a subclass does not implement all abstract methods, it will also be considered an abstract class, and you won't be able to create instances of it.
- **Object Creation:** You cannot create an instance (object) of an abstract class directly. You can only create instances of concrete subclasses that have implemented all the abstract methods.
- **Regular Methods:** Abstract classes can also contain regular (non-abstract) methods with implementations. These methods can be accessed by instances of the subclasses.

Main takeaways:

- Abstraction allows you to define a common interface for a group of related classes.
- It enforces a contract that subclasses must adhere to by implementing the abstract methods.
- Abstract classes cannot be instantiated directly, promoting the use of subclasses for concrete implementations.
- Abstraction helps in achieving modularity and maintainability in your code.
-